김민상

Server Developer

문제 분석과 가장 합리적인 해결책을 내세울 수 있는 개발자입니다.

VisualVM을 통한 **스레드덤프 분석과 로그 분석**을 통해 EntityManager 트랜잭션 오류의 근본 원인인 **커넥션 풀 고갈 문제를 정확히 진단하고**, **Redis INCR 명령어**의 원자성을 활용한 해결책을 구현하여 TPS를 10배 이상 개선했습니다.

도서 검색 시스템에서는 **EXPLAIN 분석**을 통해 **인덱스 최적화와 No-Offset 방식을 도입**하여 페이지 조회 시간을 2.6초에서 0.00044초로 단축시켰습니다.

Contact

minseusang@gmail.com

https://github.com/minsang-alt

https://babgeuleus.tistory.com/

https://evoblog.life/

SKILLS

Java Spring MySQL Redis JPA AWS

학력

한성대학교

컴퓨터공학부 , 2019.03 ~ 2025.08

프로젝트

AgileHub

백엔드 개발 담당 / 인프라 운영 2024.03 ~ 2024.06, 2024.10 ~ 진행중

주요 기여 기술: Java17, SpringBoot, MySQL 8.0, JPA, AWS, Redis, Docker, Ngrinder, VisualVM

1. **이슈 생성 API 구현 및 성능 개선**

- **대규모 조직(1000명+) 기준 이슈 생성 성능 목표 설정**
  - 피크타임 동시 사용자 50-100명 처리 목표
  - 1인당 일평균 10건 처리, 피크 부하(10배) 및 안정성(10-20배) 고려해 TPS 30-40 설정
  - 100 vUser 부하 테스트 환경 구축

- **커넥션 풀 고갈 문제 분석**
  - 부하 테스트 중 20명 vUser에서 에러율 급증 확인
  - VisualVM 스레드 덤프 분석으로 요청당 두 개의 커넥션 할당이 빠른 고갈을 일으키는 문제 발견
  - 원인: 이슈번호를 생성할 때 사용된 REQUIRES_NEW 트랜잭션 전파 속성 (비관적락 최적화를 위했음)
  - 해결: 커넥션 풀 크기를 늘리는 것이 아닌 REQURED로 변경하여 단일 커넥션 사용 구조로 개선

- **Redis 기반 이슈 번호 생성 시스템 구축**
  - 초기 문제: REQUIRED로 변경 이후, 비관적 락으로 인한 성능 저하
  - 1차 시도: 낙관적 락 + 재시도 전략 적용
    - 하지만 동시 요청 많을 때 충돌과 재시도 증가로 성능 악화
  - 분석 과정
    - EXPLAIN 분석 결과 DB 쿼리는 이미 최적화 상태
    - 근본적인 동시성 처리 방식 변경 필요성 확인
  - 최종 해결방안: Redis 기반 설계
    - Redis INCR 명령어의 단일 스레드/원자성 특징으로 락 없이 동시성 해결
    - Write-behind 전략을 도입하여 업데이트된 이슈번호를 30초마다 스케줄러로 DB에 비동기 반영
    - 안정성 확보
      - 단순 카운터 특성을 고려해 Redis Cluster 대신 Replication Sentinel 구성으로 고가용성 확보

- **HikariCP와 톰캣 스레드 풀 최적화**
  - 목표: 100 VUser 동시 요청 처리 시 에러율 0% 달성
  - 커넥션 풀 설정
    - HikariCP connection-timeout 5초 설정 (요청 폭주 시 대기 시간 확보)
    - 부하 테스트 결과 기반으로 Connection Pool 200개로 설정
  - 리소스 조정
    - VisualVM 모니터링으로 병목 지점 분석
    - MySQL max_connections 500으로 설정 (여유값 확보)
    - 톰캣 Thread Pool 300으로 설정 (Connection Pool의 1.5배, 스레드 풀 고갈 방지)

- 최종적으로 목표했던 100 vUser 환경에서 **TPS 33→365, 응답시간 2.9초→0.26초 개선**

2. <u>이메일 초대 시스템 성능 및 안정성 개선</u>

- **초대 토큰 구현 방식 개선**
  - 문제: JWT 사용 시 Base64 디코딩으로 payload 노출 위험
  - 해결: UUID v4 기반 랜덤 토큰 방식 채택
  - 성능 검증: SecureRandom 대비 생성 속도 2배 향상, 100만 건 테스트 시 충돌 없음

- **이메일 전송 시스템 개선**
  - 초기 문제: Gmail SMTP의 낮은 전송 속도, 높은 스팸 필터링율
  - 해결: AWS SES 도입으로 안정적 전송과 도달률 향상
  - 효과: SPF/DKIM 인증으로 스팸 분류 최소화, 네이버메일 지연 문제 해결

- **초대 코드 저장소 설계**
  - Redis 선택 이유
    - TTL 기능으로 10분 유효기간 자동 관리
    - 다중 서버 환경에서 원격 캐시로 활용
  - 메모리 최적화: UUID Base64url 인코딩으로 키-값 크기 **256→120바이트** 감소

- **이메일 비동기 발송 및 장애 대응 설계**
  - CompletableFuture 기반 비동기 처리로 응답시간 **578ms → 130ms로 개선**
  - 장애 대응
    - Redis 기반 상태 관리로 중복 전송 방지 (PENDING→SENDING→SENT/FAILED)
    - @Retryable로 일시적 장애 시 500ms 간격, 지수 백오프로 재시도
    - ThreadPool 설정 최적화 및 CallerRunsPolicy로 시스템 안정성 확보

3. <u>CI/CD 파이프라인 구축</u>

- **CI 도구 선정 과정**
  - Jenkins vs GitHub Actions 비교 분석 수행
  - GitHub Actions 선택 이유: 팀 학습 곡선 완화, 기존 GitHub 통합 이점
  - self-hosted runners 도입으로 보안 강화 및 커스텀 환경 구성
- Buildx와 깃헙 캐시를 활용해 **빌드 시간 15분에서 1분대로 단축**

4. <u>DB 구조 리팩토링: 다중 테이블 상속에서 단일 테이블 전략으로 개선</u>

- 문제사항: 조인 전략을 사용한 이슈 관리 시스템의 복잡성과 성능 저하 문제 해결
- 해결 방안: 단일 테이블 전략으로 테이블 구조 개선 및 코드 최적화
- **마이그레이션 과정**
  - 데이터 손실 방지를 위한 점진적 트리클 마이그레이션 전략 구현
  - 새 테이블 설계 및 200만 건 이상의 데이터 안전한 이전
  - 읽기 전용 상태 설정으로 데이터 무결성 보장 및 안전한 마이그레이션
- **성과**
  - 마이그레이션 전후 데이터 무결성 검증을 위한 테스트 케이스 구축
  - 코드 복잡성 감소 및 유지보수 용이성 증가
  - 테스트 코드 커버리지 24%에서 42%로 향상

- GitHub: <u>https://github.com/AgileHub-DQ/Backend</u>
- 웹사이트: <u>https://www.agilehub.info/</u>

BookChallenge 챌린지에 참여하면서 독서 습관을 만들어가는 플랫폼

백엔드 개발 담당 / 인프라 운영 2024.08 ~ 2024.10

주요 기여 기술: Java17, SpringBoot 3.3.2, MySQL 8.0, JPA

1. 대용량 도서 조회/검색 시스템 성능 최적화

- **성능 목표 수립 및 분석**
  - 문제: 500만 건 도서 데이터 조회/검색 시 낮은 TPS (4.3)
  - 목표: 실사용자 패턴 분석으로 목표 TPS 200 설정
    - 기본 TPS 0.23 (일평균 1000명 × 20회 조회)
    - 피크타임(10배), 동시접속(5배), 확장성(2배) 고려
  - 검증: 50 vUser 부하테스트 환경 구성

- **페이징 API 성능 개선**
  - 1차 개선: 인덱스 최적화
    - 문제: EXPLAIN 분석 결과 테이블 풀스캔/filesort 발생
    - 시도: 정렬 조건(created_at, book_name)에 단일 인덱스 적용
    - 한계: Offset 300만+ 시 실행시간 2.6초, filesort 발생
  - 2차 개선: No-Offset 방식 도입
    - 이유: 커버링 인덱스는 관리 복잡성/쓰기 성능 저하
    - 효과: 인덱스 레인지 스캔으로 전환, TPS 245 달성
    - 결과: 실행시간 **2.6초 → 0.00044초**로 개선

- **검색 API 페이징 성능 개선**
  - 1차 개선: 복합 인덱스 최적화
    - 문제: 다중 조건 검색 시 풀스캔으로 카운트쿼리 5.3초 소요
    - 분석: LIKE '%검색어%' 패턴이 인덱스 활용 저해
    - 해결: page_count, book_name 순 복합 인덱스 재구성
    - 효과: 검색시간 1초 → 0.3초로 단축
  - 2차 개선: 카운트 쿼리 최적화
    - 문제: 검색 조건별 카운트 쿼리 과다 발생
    - 해결: 검색 시 고정 페이지수 적용, 페이지 이동 시만 카운트
    - 효과: **TPS 18.9 → 219, 응답시간 528ms → 45.2ms**
  - 기술 검토 내용
    - Fulltext Search + ngram: 연속 숫자 패턴 테스트 데이터로 인한 문서 ID들의 랜덤 I/O로 제외
    - 카운트 캐싱: 동적 검색 조건으로 인한 낮은 캐시 효율성으로 제외
- GitHub: https://github.com/f-lab-edu/book-challenge

활동

우아한테크코스 프리코스 2023.11~2023.12

- 객체지향 설계 원칙을 적용하여 유연하고 확장 가능한 코드 구현
- 단위테스트 코드 작성과 함께 미션을 구현
- 리뷰를 하면서 효과적인 코드 리뷰 방법론 습득

자바스터디 2023.07~2023.11

- @GetterSetter 어노테이션 구현을 통한 Java 컴파일러 동작원리 학습
  - javac 컴파일 과정 과 AST 조작 방법 연구
  - Java Compiler API(javax.tools) 를 활용한 동적 코드 생성 기법 습득
  - 블로그글: @GetterSetter 만들기

수상

SW 캡스톤 디자인 우수상 [2024.05] 한성대학교 컴퓨터공학부